Full Stack Engineer

Technical Vetting Assignment

Instructions & Requirements

Take Home Assignment: Multi Tenant Document Workspace (4 Hour Core)

Context

You're building a minimal multi tenant "Document Workspace" for accounting firms. Each firm (tenant) has users who log in and work only with their firm's documents. Documents are simple metadata records (no file uploads) that can move through a small status lifecycle. Prevent cross tenant data access at all times.

Requirements

Build a small full stack app using Node.js + TypeScript, React, and PostgreSQL. Keep scope tight and focus on correctness, tenant isolation, and security. You may choose specific libraries and patterns.

Out of scope for this exercise:

- User registration, pagination, reporting, webhooks/external integrations, file uploads, roles/permissions beyond tenant scoping.

Task 1: Backend Service (Node.js + TypeScript + PostgreSQL)

Implement a minimal REST API that supports the workspace.

- Core tech:
  - Node.js with TypeScript
  - PostgreSQL (preferred). If PostgreSQL isn't available locally, you may use SQLite for local execution, but include PostgreSQL DDL and note any differences in SOLUTION.md.
- Domain model:
  - Tenant (organization)
  - User (belongs to exactly one Tenant)
  - Document (belongs to a Tenant) with fields including at least: identifier, tenant association, title, status, created/updated timestamps
  - Document statuses: draft, awaiting_signature, signed
- Authentication and authorization:
  - Implement login only (seed at least one user; no registration)
  - Enforce that all document endpoints restrict access to the authenticated user's tenant
  - You may use session or token based auth; ensure secure password storage
- API endpoints (minimum):
  - Auth: login
  - Documents:
    - List documents for the current tenant with a simple text search on title (choose search; do not implement pagination)
    - Update a document's status (e.g., draft ' awaiting_signature ' signed)
  - Optional if time allows:
    - Create a new document for the current tenant
- Data and operations:
  - Provide a PostgreSQL schema (DDL) and either migrations or a setup script
  - Seed script: at least one tenant, one user, and several sample documents
  - Use environment variables for configuration (e.g., DB connection, secrets)
  - Validate inputs and return clear error responses
  - Prevent injection and data leakage:

        - Use parameterized queries
        - Use a database connection pool
    - Explain tenant scoping and any performance considerations briefly in SOLUTION.md

Task 2: Frontend (React + TypeScript)

Build a minimal SPA that consumes your API.

- Screens:
    - Login screen that authenticates against your backend
    - Documents screen:
        - List documents for the current tenant
        - Simple text search on title (no pagination)
        - Change a document's status (e.g., via a small control per row)
        - Optional if time allows: Create a new document

- Behavior:
    - Manage auth state on the client and handle unauthorized responses gracefully
    - Show basic loading and error states for API calls
    - Keep the UI simple while demonstrating modern React patterns for forms and data fetching

Deliverables

1. A single GitHub repository containing:

- All the source code for the assignment
- A README.md with instructions to run locally.
- A SOLUTION.md describing the design and trade-offs you made.

2. Video Demo (5-10 minutes using Loom or similar):

- Demonstrate the working project by doing a voice over
- Walk through your code architecture
- Explain key design decisions and any trade-offs made
- Highlight interesting technical implementations

Time Estimate

This should take ±4 hours to complete, but you can take as long as you need.